El flujo es: un modelo matemático del brazo (`brazo_cinematica.py`) alimenta un entorno de simulación (`reach_env.py`); sobre ese entorno se entrena un agente (`train_ppo.py`), se analiza por qué falla o acierta (`diagnostico.py`) y se compara contra dos referencias (`comparar_agentes.py`).

Archivos y dependencias

Archivo	Función	Depende de
`brazo_cinematica.py`	Modelo del brazo (posición de la punta)	Ninguno
`reach_env.py`	Entorno de simulación	`brazo_cinematica.py`
`train_ppo.py`	Entrenamiento y evaluación	`reach_env.py`
`diagnostico.py`	Análisis de un modelo guardado	`reach_env.py`
`comparar_agentes.py`	Tabla comparativa	`reach_env.py` , `brazo_cinematica.py`
`dh_6gdl.py`	**Obsoleto** (ver al final)	Ninguno

Todos deben estar en la misma carpeta.

Instalación

```
pip install numpy gymnasium stable-baselines3
```

Stable-Baselines3 trae PyTorch, que es una descarga grande; si falla por tiempo de espera, reintentar con `--default-timeout=1000 --no-cache-dir` . `comparar_agentes.py --sin-modelo` funciona sin Stable-Baselines3.

Orden de uso recomendado

1. Referencias, sin entrenar: `python comparar_agentes.py --sin-modelo`
2. Entrenar con tolerancia amplia: `python train_ppo.py --radio 5`
3. Ver qué hace el agente: `python diagnostico.py --radio 5`
4. Apretar la tolerancia con el currículo: `python train_ppo.py --radio 3 --continuar`

`ppo_reach_radio5_periodo0.5` y luego `python train_ppo.py --radio 1.5 --continuar ppo_reach_radio3_periodo0.5`

5. Comparar el resultado final: `python comparar_agentes.py --modelo ppo_reach_radio1.5_periodo0.5 --radio 1.5`

Nombre de los modelos guardados

Cada entrenamiento guarda un archivo `.zip` cuyo nombre incluye el algoritmo, el radio y el periodo, por ejemplo `ppo_reach_radio5_periodo0.5.zip`. Si se usa `--delta`, el nombre lleva `delta` en lugar de `periodo`. Así una corrida no sobrescribe a otra; `--salida` permite poner otro nombre.

Conceptos mínimos de RL

- **Episodio:** un intento completo, desde una pose inicial hasta alcanzar el objetivo o agotar los pasos.
- **Observación:** lo que el agente «ve» en cada paso.
- **Acción:** lo que el agente decide (cuánto mover cada articulación).
- **Recompensa:** los puntos que recibe en cada paso; el agente aprende a conseguir más.

brazo_cinematica.py

Para qué sirve

Calcula dónde queda la punta del brazo (x, y, z) a partir de los seis ángulos de las articulaciones. Es la «caja negra» geométrica sobre la que se apoya toda la simulación.

Cómo funciona

- Usa la tabla Denavit-Hartenberg (DH) de la tesis del brazo (Tabla 9): cada eslabón aporta una matriz 4x4 y el producto de las seis da la pose del efector final. La articulación 3 lleva un desfase de −90°.
- Funciones principales: `forward_kinematics(q)` (matriz completa), `end_effector_position(q)` (solo x, y, z) y `joint_positions(q)` (origen de cada eslabón, útil para graficar).
- Los ángulos de entrada van en **radianes**. Las unidades de longitud son las de la tesis (probablemente centímetros).
- La articulación 6 no cambia la posición de la punta, solo su orientación.
- Al ejecutarlo directamente (`python brazo_cinematica.py`) comprueba que, con todos los ángulos en cero, se obtiene la posición (103.3, 9.5, 60.5) y la orientación que reporta la tesis.

Detalles importantes

- De la tesis se deducen tres datos: l1 = 60.5, l2 = 9.5 y l3 + l4 + l5 = 103.3.
- `L3 = 45` y `L4 = 40` **son valores provisionales.** Falta medir cómo se reparte esa suma en el brazo real; `L5` se calcula para conservarla. Si se corrigen, la simulación cambia y hay que **volver a entrenar**.
- Se asume que los ángulos del modelo son los mismos que usa la tesis en sus límites de giro; la validación con el brazo físico lo confirmará.
- Reemplaza a `dh_6gdl.py`, que usaba los parámetros del código MATLAB, los cuales no coinciden con la tesis.

reach_env.py

Para qué sirve

Define el «videojuego» en el que practica el agente: un brazo virtual de 6 articulaciones que debe llevar la punta a un objetivo. Sigue la interfaz estándar de Gymnasium (`reset()` y `step()`), así que las librerías de RL lo entienden directamente.

Cómo funciona

Observación (15 números): los 6 ángulos normalizados entre −1 y 1 respecto al centro de su rango, la posición de la punta, la posición del objetivo y el vector de la punta al objetivo (las posiciones divididas entre 100).

Acción (6 números entre −1 y 1): cada uno es la fracción del giro máximo por paso de esa articulación. El giro máximo es velocidad por periodo de control. Las velocidades son las del modo manual del Arduino (J1 7.5, J2 4.5, J3 7.2, J4 5.6, J5 16.4 y J6 66.7 grados por segundo). Con el periodo por defecto de 0.5 s, el giro máximo por paso es 3.75°, 2.25°, 3.6°, 2.8°, 8.2° y 33.35°. Una acción continua equivale a mover la articulación a su velocidad fija durante esa fracción del periodo, que es lo que hace el control manual del brazo.

Límites de giro (Tabla 15 de la tesis): J1 −270° a +90°, J2 ±60°, J3 ±135°, J4 ±180°, J5 ±70° y J6 ±90°. Los ángulos se cortan en esos límites.

Inicio de cada episodio (`reset`):

1. Se sortea una pose aleatoria usando el 80% del rango de cada articulación alrededor de su centro; la posición de la punta en esa pose es el **objetivo**. Así el objetivo siempre es alcanzable.
2. El brazo arranca en esa misma pose desplazada hasta `start_spread_deg` grados por articulación (30° por defecto) o, si es `None`, en una pose totalmente aleatoria (la versión más difícil).
3. Si la punta queda a menos de tres veces el radio de éxito del objetivo, se vuelve a sortear (hasta 20 intentos).

Recompensa en cada paso: `−distancia/100 − 0.01 × (promedio de la acción al cuadrado)`, más un bonus de 10 al llegar. Cada paso lejos del objetivo cuesta puntos, así que conviene llegar rápido; el castigo a la acción desalienta los movimientos bruscos.

Fin del episodio: termina con éxito si la punta queda a menos de `success_radius` (1.5 por defecto) y se corta sin éxito a los 200 pasos. En `info` se devuelven la distancia y si hubo éxito.

Parámetros del entorno

Parámetro	Valor por defecto	Qué controla
`control_period_s`	0.5	Duración de cada paso, en segundos
`max_delta_deg`	ninguno	Si se indica, giro máximo uniforme por paso (grados), en lugar de velocidad por periodo
`success_radius`	1.5	Distancia para dar el objetivo por alcanzado
`max_steps`	200	Pasos máximos por episodio (100 s de brazo con el periodo

		por defecto)
`action_penalty`	0.01	Peso del castigo a movimientos bruscos
`success_bonus`	10	Premio por llegar
`start_spread_deg`	30	Dificultad del inicio (currículo)

Además ofrece `nombre_modelo(...)`, que construye el nombre de los modelos guardados y que usan los otros scripts.

Detalles importantes

- Es un modelo **solo cinemático**: no incluye inercia, fricción, holguras ni retardos. Las diferencias entre simulación y brazo real se medirán más adelante.
- La pinza no está modelada. La articulación 6 (servo) se trata igual que las demás, aunque no afecta la posición de la punta.
- Cualquier cambio en el entorno (longitudes, límites, velocidades, recompensa) hace que los modelos guardados dejen de ser válidos: se pueden cargar, porque el tamaño de la observación y de la acción no cambia, pero se comportarán distinto. Conviene reentrenar.
- Los límites de giro vienen de la tesis y las velocidades de `Codigo_final.ino`; ambos deben revisarse en el brazo real.

train_ppo.py

Para qué sirve

Entrena al agente, lo evalúa y guarda el modelo. Aunque se llama `train_ppo.py`, también entrena con SAC.

Cómo funciona

1. Crea el entorno de entrenamiento (con registro de episodios) y otro de evaluación con los mismos parámetros.
2. Mide la **línea base**: un agente que actúa al azar, en 100 episodios.
3. Crea el modelo (red neuronal `MlpPolicy` de Stable-Baselines3 con sus hiperparámetros por defecto) o, con `--continuar`, carga uno ya guardado y muestra cómo le va antes de seguir.
4. Entrena durante `--steps` pasos.
5. Evalúa el modelo entrenado en 100 episodios sin ruido de exploración y con semillas fijas (1000 a 1099), e imprime el porcentaje de éxito, la distancia final media y los pasos medios.
6. Guarda el modelo en un `.zip`.

Opciones

Opción	Valor por defecto	Qué hace
`--algoritmo`	`ppo`	`ppo` o `sac`

`--steps`	500000	Pasos de entrenamiento
`--seed`	0	Semilla del entrenamiento
`--salida`	automático	Nombre del modelo guardado
`--radio`	1.5	Tolerancia de éxito
`--dispersion`	30	Grados por articulación que el inicio se aleja de una pose que alcanza el objetivo
`--inicio-aleatorio`	desactivado	Inicio totalmente aleatorio (ignora `--dispersion`)
`--periodo`	0.5	Duración de cada paso, en segundos
`--delta`	ninguno	Giro uniforme por paso, en grados (ignora las velocidades reales)
`--continuar`	ninguno	Nombre (sin `.zip`) de un modelo guardado desde el cual seguir entrenando

Detalles importantes

- **Currículo de tolerancia:** entrenar con `--radio 5` y seguir con `--radio 3` y `--radio 1.5` usando `--continuar`. El algoritmo indicado debe ser el mismo con el que se guardó el modelo.
- Con `--continuar` el contador de pasos no se reinicia.
- Los hiperparámetros no están ajustados. Los valores por defecto de PPO en Stable-Baselines3 son: tasa de aprendizaje 0.0003, 2048 pasos por actualización, lotes de 64, 10 épocas y gamma 0.99; pueden variar entre versiones de la librería.
- No hay normalización adicional de observaciones ni de recompensas.
- La línea base aleatoria depende de la configuración (con una tolerancia mayor, el azar acierta más); la referencia correcta es siempre la línea `[Agente aleatorio]` que imprime cada corrida.
- En el equipo del proyecto, 500 mil pasos tardaron entre 6 y 11 minutos; SAC tarda más por paso que PPO.
- Durante el entrenamiento, `ep_rew_mean` (recompensa media) debe subir y `ep_len_mean` (pasos por episodio) debe bajar. Conviene guardar lo que se imprime para armar las curvas del informe.

diagnostico.py

Para qué sirve

Explica por qué un modelo ya entrenado no llega al objetivo: si pasa cerca y se pasa de largo, si se queda quieto en un punto o si anda lejos.

Cómo funciona

Carga el modelo, corre `--episodios` episodios (100 por defecto) con semillas fijas y registra la distancia en cada paso. Imprime:

- la distancia inicial, final y **mínima alcanzada** medias (si la mínima es mucho menor que la final, el agente pasa cerca pero no se queda);
- el porcentaje de episodios en que la punta llegó, en algún momento, a menos del radio de éxito y a menos de 5, 10 y 20;
- la distancia final media solo en los fallos;
- la distancia al objetivo cada 20 pasos en tres episodios, para ver si oscila, mejora o se estanca.

Detalles importantes

- **Debe usarse con los mismos parámetros con los que se entrenó el modelo** (`--radio`, `--periodo` o `--delta`, `--dispersion`). Si no, el nombre del modelo no coincide o el entorno es distinto.
- Para un modelo con otro nombre, `--modelo nombre_sin_zip`.
- El éxito se calcula como «la distancia mínima fue menor que el radio», que coincide con la definición del entorno porque el episodio termina al llegar.
- No entrena ni modifica nada; es seguro correrlo las veces que haga falta.

comparar_agentes.py

Para qué sirve

Produce la tabla comparativa para el informe: agente aleatorio, controlador clásico y agente de RL sobre los **mismos episodios**.

Cómo funciona

- **Agente aleatorio:** acciones al azar; es la línea base mínima.
- **Controlador clásico:** no aprende; en cada paso calcula con el Jacobiano del brazo (por diferencias finitas) y su pseudoinversa amortiguada qué movimiento acerca la punta al objetivo. Es el techo práctico contra el que se mide al agente de RL.
- **Agente de RL:** el modelo guardado, sin ruido de exploración. Se incluye siempre que no se use `--sin-modelo`, cargando el modelo indicado con `--modelo` o el de nombre por defecto.
- Corre cada uno en los mismos 100 episodios (semillas desde 1000) e imprime el porcentaje de éxito, la distancia final media y los pasos medios de los episodios exitosos. Guarda el detalle por episodio en `comparacion_agentes.csv` (método, episodio, distancia inicial, distancia final, pasos y éxito).

Detalles importantes

- `--sin-modelo` compara solo aleatorio y clásico y no requiere Stable-Baselines3.
- Usar los mismos parámetros que en el entrenamiento (`--radio`, `--periodo` o `--delta`, `--dispersion`, `--algoritmo`).
- Referencias medidas con el modelo actual, con radio de 1.5 y dispersión de 30°: el agente aleatorio acierta en alrededor del 4% de los episodios y el controlador clásico en el 100%. Con inicio totalmente aleatorio el clásico baja a cerca de la mitad, porque se atora en los límites de giro.
- El controlador clásico usa el modelo del brazo (el mismo del entorno), así que no sirve como prueba independiente del modelo.

dh_bgdl.py (obsoleto)

Port a Python de la cinemática del código MATLAB del Anexo IV de la tesis. Sus parámetros no reproducen la pose home que reporta la propia tesis, así que no representa al brazo real. Fue sustituido por `brazo_cinematica.py` y ya no lo usa ningún script.

Limitaciones generales

- El agente se entrena en simulación y no controla el brazo físico: el código del Arduino no acepta comandos desde la computadora.
- Las longitudes `L3` y `L4`, y los límites y velocidades del entorno, deben validarse con el brazo real.
- Los scripts de lectura de encoders (`lector_encoders.py`, `serial_to_csv_encoder.py` y el sketch del Arduino monitor) están documentados aparte y no intervienen en el entrenamiento.